Отчёт по практике 8

Безопасность контейнеров: веб-приложение, база данных и сканирование

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Что требовалось

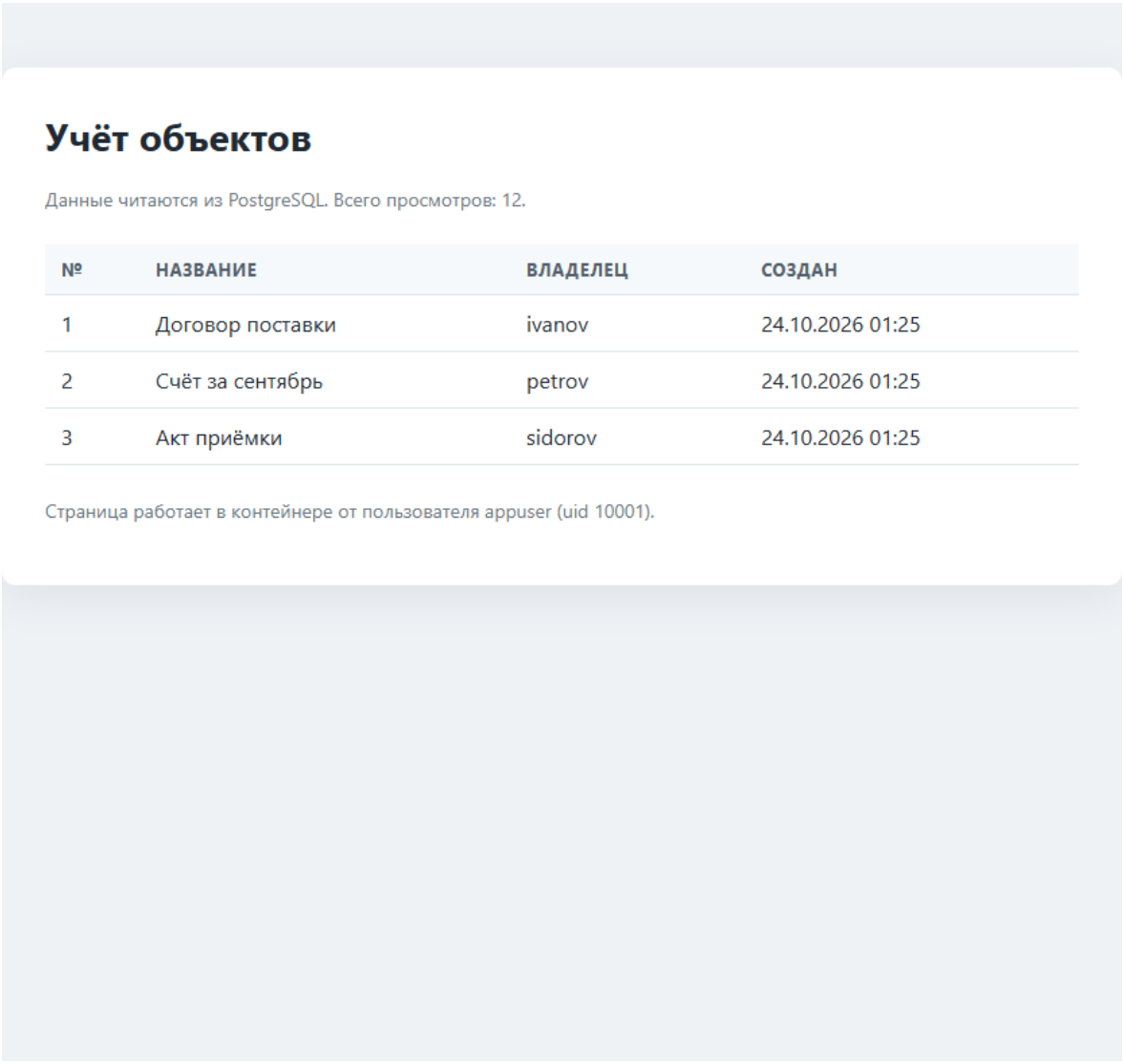
Нужно было развернуть веб-сайт с базой данных, настроить постоянное хранение и отдельное сетевое окружение, а затем проверить контейнер и образ средствами безопасности. В отчёте требуется привести скриншоты, разобрать вывод сканеров и предложить меры по устранению находок.

2. Архитектура стенда

Стенд состоит из двух контейнеров. p8-web — Flask-приложение на Python 3.12, которое отдаёт страницу со списком объектов и обращается к PostgreSQL. p8-db — официальный postgres:16-alpine. Приложение опубликовано только на 127.0.0.1:8088; база не публикует порт наружу и доступна приложению по имени db внутри пользовательской bridge-сети p8_backend. Данные лежат в именованном томе p8_pgdata, поэтому переживают пересоздание контейнеров.

Секреты не зашиты в Dockerfile: пароль базы передаётся через .env (в репозиторий попадает только .env.example) и переменную DATABASE_URL. Для учебного стенда пароль фиксированный, для реального его нужно генерировать и хранить в Docker secrets или другом хранилище.

Компонент	Образ / параметры	Назначение
p8-web	p8-web:local, 198 МБ, uid 10001	Flask + gunicorn, страница объектов
p8-db	postgres:16-alpine, 420 МБ	PostgreSQL, том p8_pgdata
Сеть	p8_backend, bridge	изоляция от default bridge
Порт	127.0.0.1:8088 → 8080	доступ только с хоста
Том	p8_pgdata	постоянное хранение данных



Скриншот страницы: данные приходят из PostgreSQL, счётчик просмотров увеличивается при каждом обновлении.

3. Запуск и проверка

Последовательность запуска:

```
copy .env.example .env
docker compose up -d --build
docker compose ps
curl http://127.0.0.1:8088/
```

Оба контейнера переходят в состояние healthy. Приложение проверено запросом к / и к /healthz, база — через healthcheck pg_isready. На скриншоте ниже — фактический вывод docker compose ps.

```
docker compose ps
NAME      IMAGE                COMMAND                  SERVICE   CREATED          STATUS
             PORTS
p8-db     postgres:16-alpine   "docker-entrypoint.s…"   db        15 minutes ago   Up 15 minutes (he
    althy)   5432/tcp
p8-web    p8-web:local         "gunicorn --bind 0.0…"   web       2 minutes ago    Up 2 minutes (hea
    lthy)    127.0.0.1:8088->8080/tcp
```

Постоянное хранение проверено так: до перезапуска счётчик показывал 3 просмотра, после docker compose restart — 4, после docker compose down и up без удаления тома — 5. Данные и схема остались на месте, потому что том p8_pgdata не удалялся. Команда docker compose down -v удалила бы том и обнулила базу — это ожидаемое поведение, но в отчёте важно не перепутать down и down -v.

4. Настройки безопасности контейнеров

Мера	Где	Зачем
Непривилегированный пользователь	Dockerfile, USER 10001	приложение не root
read-only корневая ФС	web и db	нельзя дописать файлы в образ
tmpfs /tmp	web	gunicorn может писать временные файлы
cap_drop: ALL	web	убирает лишние capabilities
no-new-privileges	web и db	запрещает повышение привилегий
Лимиты CPU/памяти/PIDs	compose	защита от исчерпания ресурсов
Порт только на 127.0.0.1	web	сайт не доступен всей сети
Отдельная сеть	p8_backend	нет доступа к default bridge
HEALTHCHECK	Dockerfile и compose	видно, что сервис жив

Отдельно про базу: официальный postgres при первом старте запускается от root, чтобы выставить владельца каталога данных, а затем сам сбрасывает права на пользователя postgres. Поэтому в Docker Bench он отмечен как «Running as root». Это особенность официального образа, а не приложения; на реальном стенде лучше использовать заранее подготовленный том и запуск от uid postgres.

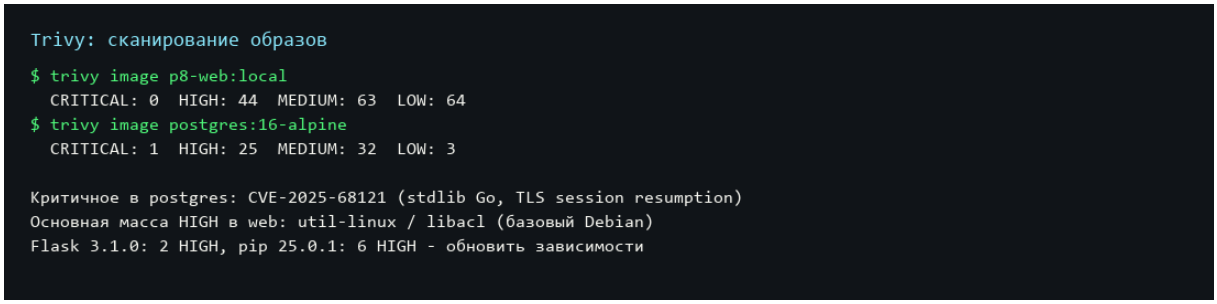
5. Trivy: сканирование образов

Образы сохранены в tar и просканированы Trivy без доступа к Docker-сокету:

docker save p8-web:local -o p8-web.tar
docker run --rm -v "${PWD}:/work" aquasec/trivy image --input /work/p8-web.tar

Образ	CRITICAL	HIGH	MEDIUM	LOW	UNKNOWN	Всего
p8-web:local	0	44	63	64	4	175

postgres:16-alpine	1	25	32	3	3	64



В p8-web основная масса HIGH приходится на базовый Debian: util-linux, libacl и другие системные пакеты. Из прикладных зависимостей выделяются Flask 3.1.0 (2 HIGH) и pip 25.0.1 (6 HIGH). В postgres:16-alpine один CRITICAL — CVE-2025-68121 в Go stdlib (crypto/tls, неправильная проверка сертификата при возобновлении TLS-сессии). База не публикует порт наружу, поэтому этот путь не доступен извне, но образ всё равно нужно обновлять.

Быстрых «патчей в Dockerfile» здесь нет: часть уязвимостей без исправлений в текущем дистрибутиве. Правильные меры — пересобирать образ регулярно, перейти на более свежий базовый образ (slim-bookworm или distroless), обновить Flask и убрать pip из runtime-слоя, а также добавить Trivy в CI с порогом по CRITICAL/HIGH. Для postgres — обновлять тег и следить за релизами образа; критичный TLS-баг не влияет на текущую схему, пока база недоступна по сети.

6. Docker Bench Security

Docker Bench запущен из официального репозитория на хосте (в контейнере на Windows он не видит docker.sock). Проверок 117, итоговый score — 3. Много замечаний относится к самому Docker Desktop: аудит хоста, user namespaces, AppArmor и централизованный журнал на Windows не настроены. По стенду p8 после внесённых исправлений остались:

Замечание	Отношение к p8	Что делать
Running as root: p8-db	официальный postgres	запускать от uid postgres после init
No AppArmorProfile: p8-web/p8-db	Docker Desktop	включить AppArmor на Linux-хосте
Port in use: 8088	опубликован только loopback	оставить, это нужный порт
No Healthcheck found	исправлено	HEALTHCHECK добавлен в Dockerfile
No CPU restrictions	исправлено	cpus: 0.50 в compose
Root FS mounted R/W: p8-db	исправлено	read_only: true и tmpfs



На скриншоте уже финальный прогон: замечаний про healthcheck и CPU-лимиты там нет. В таблице они оставлены с пометкой «исправлено», чтобы было видно, что именно изменилось после первого сканирования. Полный текстовый вывод лежит в results/docker_bench_clean.txt.

7. Что исправлено по итогам сканирования

После первого прогона Docker Bench в образ добавлен HEALTHCHECK, в compose — лимиты CPU и read-only корневая ФС для базы с tmpfs для /tmp и /var/run/postgresql. Повторный прогон подтвердил, что замечания про healthcheck, CPU и R/W root для p8-db ушли. Остались только root-процесс postgres, отсутствие AppArmor на Docker Desktop и опубликованный loopback-порт.

Дополнительные меры, которые имеет смысл сделать в реальном проекте: перейти на distroless или alpine-базу, убрать pip из финального слоя, подписать образ (cosign/DCT), включить сканирование Trivy в CI с блокировкой по CRITICAL, использовать Docker secrets вместо .env, настроить AppArmor и централизованный сбор логов, ограничить исходящие соединения контейнера.

8. Выводы

Стенд из веб-приложения и PostgreSQL развёрнут, данные лежат в именованном томе и переживают перезапуск, сеть изолирована, наружу открыт только loopback-порт. Контейнер приложения работает от uid 10001 с read-only корневой ФС, без capabilities и с no-new-privileges. Trivy нашёл 175 уязвимостей в p8-web и 64 в postgres, из них одна критическая в Go stdlib postgres; разобраны причины и предложены меры. Docker Bench дал 113 предупреждений по хосту и образу, по p8 после исправлений остались только объяснимые замечания.

Главный вывод: сканеры показывают не «плохой/хороший контейнер», а набор компромиссов. Образ на официальной базе всё равно содержит десятки CVE, которые нельзя исправить в Dockerfile; помогает регулярная пересборка, минимизация образа и перенос проверок в CI. Ограничение работы — Docker Desktop на Windows, поэтому часть проверок хоста (AppArmor, аудит, user namespaces) недоступна и отмечена как среда, а не как дефект стенда.